ELEC 278 – Data Structures and Algorithms

Lab 5: Hash Tables, Graphs

November 15 – November 25

Contents

1. Basic Lab Information 2
   1.1 New Instructions for this lab 2
   1.2 Overview and Background 2
2. Lab Resources 2
   2.1 List of Files provided with this Lab 3
3. Lab Objective 3
   3.1 Topics covered in this lab 3
4. Lab Steps 3
   4.1 Hash Tables 3
   4.2 Graph Search 4
   4.3 Submission Instruction 6

Copyright 2021 David Athersych. This material is provided for use of students registered in ELEC278 at Queen's University, Kingston in the Fall semester 2021.

Document History

Date	Author	Description
2020-10-25	DFA	Complete rewrite

1. Basic Lab Information

You should review the Lab Overview document prior to reading the material for this lab exercise.

If you have already installed a C compiler on your computer (or have one available), then you can continue with the lab. If you have not installed the C programming tools, please stop working on this lab, and review Lab Overview document Appendix A. You may also wish to review Tutorial 1 before you attempt this lab.

Be sure to note the instructions for submitting your work. Note that it is acceptable that you submit more than one version of your work. ***In the absence of any indication to the contrary***, the markers will only look at your latest submission.

You may also find it useful to review previous labs as you work on this lab exercise.

1.1 New Instructions for this lab

As announced on onQ, you will be permitted to work with one partner on Lab 5. You may find it more instructive to work with someone to discuss ideas with. Both partners are expected to contribute equally to the work handed in and both partners will be awarded the same grade for the work.

The work handed in by two-person lab team **MUST** include the names of both partners. **BOTH PARTNERS WILL SUBMIT A COPY OF THE LAB WORK.** The markers will review one of the copies and will award both team members the same mark.

Carefully note the instructions for submitting lab work, found in Section 4.3

1.2 Overview and Background

This lab exercise is designed to help learners become more familiar with hash tables discussed in week 11 and the graph algorithms discussed in weeks 11 and 12 of ELEC278.

2. Lab Resources

Specific to this lab:

1. Lab Instructions (this document)
2. A 2 zip files containing source code (called GraphCode_BFS.zip and HashCode.zip).

Learner provided:

1. Programming environment selected by the learner. The Lab Overview document provides a list

of possible C programming tools that you may use.

This lab has code for you to read. The purpose of the example code is to reinforce what you have seen in the class material. It also provides you with code to cut-and-paste into the code you write to solve the assigned problems. This lab has multiple programming problems to solve.

2.1 List of Files provided with this Lab

(See the files GraphCode_BFS.zip and HashCode.zip contents)

3. Lab Objective

There are two objectives for this lab:

1) Review and correct the hash table code provided.
2) Review the output from various graph search routines.

3.1 Topics covered in this lab

- Hash Tables.
- Graph searches.

4. Lab Steps

This lab includes several steps, all of which involve using the code provided to you in the zipped directory**. Note there are two zip files – one for the hash table code and one for the graph code.**

Before you get down to the deliverables, examine all the source files. Confirm that you can compile the programs supplied.

4.1 Hash Tables

The code provided for working with hash tables inserts, using chaining, about 60 items of data – strings – into a hash table with 40 entries. What this means is that there are 40 distinct hash indices [0..39] and that every data item has been mapped to one of those indices.

This is a two-step process. First, a hash code is generated. The formula used to compute the hash code is:

$$h(s) = \sum_{i=0}^{n-1} S[i].\,31^{n-1-i}$$

where S[i] means the character at index i. The term is the ith character multiplied by 31 to the

power n-1-i. (You can judge the goodness of this formula as you explore the behavior. (Do not be too concerned about the details of the hash function – it is just an example of something a little more elaborate than what was used in class.)

The hash code is then taken mod 40 (the table size), to get the actual index used.

You should review the provided code – hash.c – compile it and run it. You will see that it reads a word of data from a file, inserts the word into the hash table, displays the table, and then waits for the user to type return before it does another word. The tables displayed show the chains for each hash index. At the end, it shows the final hash table.

You will note that the output is not correct – the value for number of entries is not correct.

Deliverable – Hash Table

1. Fix the hash program code so that it displays the correct number of entries. Use this information to compute and display the load factor as well.
2. Add code after all the words have been read in so that the average chain length (ACL) can be computed. The average chain length is determined by summing the length of every **non-zero length** chain and dividing by the number of non-zero length chains. (This would be a measure of clustering.) Include this figure in the output shown after all the words have been added to the table.
3. By trial and error, determine what **size table** will give an ACL of less than 2, for the set of data you have been given. The code you submit should have a hash table which is the size you determine.
4. Submit this deliverable as one file titled L5_HASH.C. The file must contain the name or names of the people who worked on the code, in the format Last Name, First Name as the first line in the code. It would be helpful if the names were provided in sorted order. If you have worked in a group of two, then BOTH students **must submit identical copies of the file**.

4.2 Graph Search

The code provided to perform a Breadth-First Search on a graph works on fixed data built into the program. The program determines shortest paths from a starting vertex to all other vertices and does it for all vertices. It reports whether starting at a particular vertex results in some vertices being inaccessible. Review the code and make sure you understand what the code is doing. Note that there are four different situations – three with plain edges and one with weighted edges.

As noted above, the program used data that is static – built into the code. The program would be more versatile if the data were read from a file. In the class videos and slides, there was a description of what the file would look like:

First line holds number, N, of vertices.

Next N lines contain the names of the vertices. In the case of this code, the names are single letters.

The next lines contain edge information. You could handle all situations by having one line for a directed connection, with a weight; for example: A B 20 could mean an edge from A to B with a weight of 20. If an edge is bidirectional, there would be two lines: A B 20 and B A 20 – to represent an undirected edge of weight 20 between A and B. This list could end with a sentinel of some kind - maybe an edge weight of -1, or you could end by detecting the end-of-file. A file then could look like:

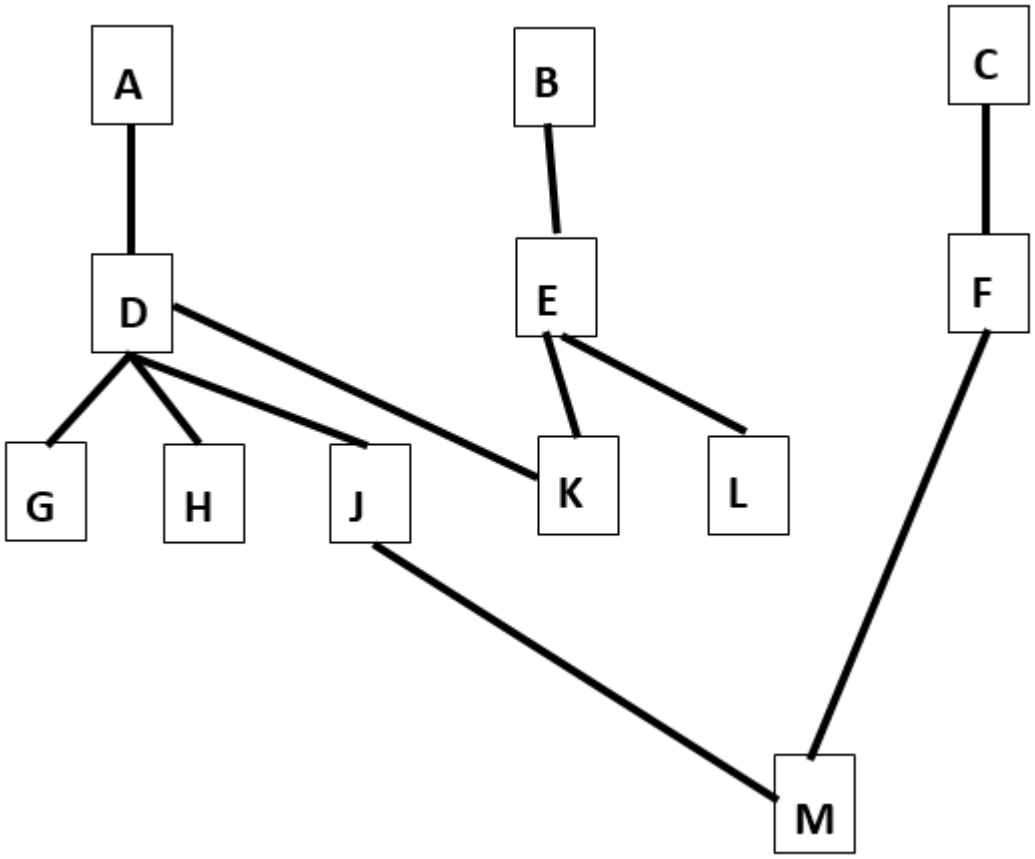
```
5
A
B
C
D
E
A B 1
B C 1
C D 1
D E 1
E A 1
```

which would represent a simple graph with one cycle.

Deliverable – Breadth-First Graph Search

1. Modify the code to read graph data from a file. Build a description for the graph in the following diagram. Run BFS on this data.
2. Now suppose edges AD, BE and CF are unidirectional – that is, there is an arrow from A to D but not from D to A. Change your description and rerun BFS. What changes?
3. Now modify BFS so that you can read in the graph description, but then your program prompts you to remove one or more edges via command line prompt and user entry. After an edge is removed, recompute the paths. Note how the removal of a line may create an island.
4. Submit this deliverable as one code file titled L5_BFS.C. The code file must contain the name or names of the people who worked on the code, in the format Last Name, First Name as the first line in the code. It would be helpful if the names were provided in sorted order. If you have worked ina group of two, then BOTH students **must submit identical copies of the file**.

(Aside: This problem is a very simplified version of power, water, or gas distribution networks. Imagine that vertices A, B and C were the vertices where power (or water or gas) were introduced into the local distribution network. Your program is a simplified version of trying to assess which endpoints no longer have supply if one of the edges – a supply line – is damaged. In a more elaborate model, the edges would have some attributes like capacity and the process of determining an alternate supply path would have to include capacity calculations.)

4.3 Submission Instruction

1- Make sure you write your name in addition to your partner name inside the submitted files.
2- Each student should submit a copy of her/his group separately for marking.
3- Place all your source codes and necessary files in a folder named in the following format:
   278-1234-Lab5 where 1234 stands for the last 4 digits of your student ID,
   e.g.: For student with ID 20196072, the folder should be named 278-6072-Lab5.
4- Compress the above folder using Zip (the extension must be .zip).
5- Upload this zipped file into the drop box of Lab5.